MixBytes()

BarterDAO Superposition Security Audit Report

MARCH 10, 2026

Table of Contents

1. Introduction	**2**
1.1 Disclaimer	2
1.2 Executive Summary	2
1.3 Project Overview	4
1.4 Security Assessment Methodology	7
1.5 Risk Classification	9
1.6 Summary of Findings	10
2. Findings Report	**11**
2.1 Critical	11
C-1 Double Order Attack via Callback Mechanism	11
2.2 High	13
H-1 Replay Attack via Balance-Based Nonce	13
2.3 Medium	14
M-1 Silent Truncation in Permit2 Transfers	14
M-2 Malicious Signer Can Create Any Order and Spend Approvals	15
M-3 DoS Attack via Dust Executions	16
M-4 No Way to Cancel Pending Oracle Change	17
2.4 Low	18
L-1 Approval Source Ambiguity	18
L-2 Precision Loss in Partial Swaps	19
L-3 Minor Issues	20
L-4 Non-Standard EIP-712 Encoding	21
L-5 Zero Staleness Allows Order Invalidation	22
L-6 Signer Rotation Invalidates Outstanding Orders	23
3. About MixBytes	**24**

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Superposition is a decentralized order book protocol that allows users to create and fill orders for token swaps. The protocol consists of a SuperpositionVault that handles order execution and a SuperpositionRouter that provides a simplified interface for order filling with callback functionality.

In this audit, we focused on examining attack vectors related to signature validation, order execution logic, replay attacks, reentrancy vulnerabilities, callback mechanisms, and token transfer operations.

We also went through our detailed checklist, covering other aspects such as business logic, common ERC20 issues, interactions with external contracts, integer overflows, reentrancy attacks, access control, typecast pitfalls, rounding errors and other potential issues.

The audit identified vulnerabilities related to replay attack protection, reentrancy in callback mechanisms, and precision loss in order calculations.

Key notes and recommendations:

- **The audit assumes that SuperpositionRouter should not retain any funds or approvals after (and before!) any transaction completion, as otherwise anyone could withdraw them later.**
- SuperpositionRouter is planned to be used in a transaction flow: `Barter Facade -> Executor -> SuperpositionRouter -> SuperpositionVault`. On Facade, minAmounts of all output tokens are checked, so if a swap on the router doesn't go as planned, the entire transaction will revert.
- The protocol does not support fee-on-transfer tokens.
- In `swap()`: the maker may receive 1 wei less than expected due to gas optimization.
- Signers may be set to zeros, effectively pausing the protocol.
- If 2/3 owners are compromised, the attacker can set an arbitrary oracle and signers and spend maker approvals at any price by signing orders with arbitrary data.
- For a public mempool, order execution is vulnerable to griefing attacks: since `filledTakerAmount` is not signed with the order, any caller can frontrun pending orders and execute them with minimal fill amounts. This attack vector is not relevant when transactions are routed through a private mempool.
- Duplicate signer addresses are not validated: when executing a `Signers` proposal via `SuperpositionVault._execute()`, `newSigners` are not checked for duplicates. If `signer[0] == signer[1]`, one key can sign for both positions, reducing the 3-of-3 scheme to an effective 2-of-3 (or 1-of-3 if all three are identical).

- allowlistEnabled in UserConfig is written in SuperpositionVault.setUserConfig() but never checked in swap() or _validateOraclePrice(). There is no allowlist enforcement logic.
- In UserConfig, a global oracle deviation is specified, but for different tokens with different volatility, a token-specific deviation may be required.
- SuperpositionVault approve() and _approve() have identical bodies. The public approve() should simply delegate to _approve().
- signatureType is not included in the signed order hash: Signature.Type signatureType is part of SuperpositionOrderPayload but is absent from ORDER_TYPE_HASH and OrderHelper.hashSingleOrder(). While not exploitable (changing the type breaks ecrecover), it violates the principle of signing everything that affects execution.
- A high-severity bug was discovered in Solidity compiler versions 0.8.28 through 0.8.33 affecting contracts using delete on transient storage when compiled with --via-ir. The project is **not affected** (uses via_ir = false and no transient storage), but we still recommend upgrading to Solidity 0.8.34+

1.3 Project Overview

Summary

Title	Description
Client Name	BarterLab
Category	DEX Aggregator
Project Name	Superposition
Type	Solidity
Platform	EVM
Timeline	18.09.2025–10.03.2026

Scope of Audit

File	Link
src/Errors.sol	src/Errors.sol
src/SuperpositionVault.sol	src/SuperpositionVault.sol
src/SuperpositionRouter.sol	src/SuperpositionRouter.sol
src/MinimalProxy.sol	src/MinimalProxy.sol
src/Denominations.sol	src/Denominations.sol
src/IPermit2.sol	src/IPermit2.sol

Versions Log

Date	Commit Hash	Note
18.09.2025	8dfd5bceb02bef6892a574da6862f673a5ea66b6	Initial Commit
03.11.2025	c3611a76302857f061ee74c860bdf872f8f933b2	Commit for re-audit

Date	Commit Hash	Note
04.11.2025	700d53240381c6757d7374bf50e789b545c44494	Commit for re-audit
19.02.2026	d02c606fd6ed5b78d7899c03dc10b11ba84bc6fa	Commit for re-audit
05.03.2026	df7a1ba0cfe895e7d0a81d5d509f975ab0f7cc89	Commit for re-audit
06.03.2026	5a59556fc8c7ef08e3541a5a3aaeddfe9dd13c76	Commit for re-audit
06.03.2026	8da54dd8c0d46f202bee9b3f5eb189c8cfa6f8da	Commit for re-audit
10.03.2026	8aad971acd35afc57df4a3840b3aea22fe1c2d7b	Commit for re-audit

Mainnet Deployments

Commit 700d53240381c6757d7374bf50e789b545c44494

File	Address	Blockchain
SuperpositionVault.sol	0x8c649e9378fb6707b014714314d4e15ac297a6bf	Ethereum
SuperpositionRouter.sol	0x0b7250866f0b014e6983cacc5b854eea7a3d9188	Ethereum

Commit 8aad971acd35afc57df4a3840b3aea22fe1c2d7b

File	Address	Blockchain
SuperpositionVault.sol	0x69355223a0ce30aee41d353387c3082e5aafc4da	Ethereum

1.4 Security Assessment Methodology

Project Flow

Stage	Scope of Work
Interim audit	**Project Architecture Review:** • Review project documentation • Conduct a general code review • Perform reverse engineering to analyze the project's architecture based solely on the source code • Develop an independent perspective on the project's architecture • Identify any logical flaws in the design OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROJECT AND IDENTIFY POTENTIAL SECURITY RISKS.
	Code Review with a Hacker Mindset: • Each team member independently conducts a manual code review, focusing on identifying unique vulnerabilities. • Perform collaborative audits (pair auditing) of the most complex code sections, supervised by the Team Lead. • Develop Proof-of-Concepts (PoCs) and conduct fuzzing tests using tools like Foundry, Hardhat, and BOA to uncover intricate logical flaws. • Review test cases and in-code comments to identify potential weaknesses. OBJECTIVE: IDENTIFY AND ELIMINATE THE MAJORITY OF VULNERABILITIES, INCLUDING THOSE UNIQUE TO THE INDUSTRY.
	Code Review with a Nerd Mindset: • Conduct a manual code review using an internally maintained checklist, regularly updated with insights from past hacks, research, and client audits. • Utilize static analysis tools (e.g., Slither, Mythril) and vulnerability databases (e.g., Solodit) to uncover potential undetected attack vectors. OBJECTIVE: ENSURE COMPREHENSIVE COVERAGE OF ALL KNOWN ATTACK VECTORS DURING THE REVIEW PROCESS.

Stage	Scope of Work
	Consolidation of Auditors' Reports: · Cross-check findings among auditors · Discuss identified issues · Issue an interim audit report for client review OBJECTIVE: COMBINE INTERIM REPORTS FROM ALL AUDITORS INTO A SINGLE COMPREHENSIVE DOCUMENT.
Re-audit	**Bug Fixing & Re-Audit:** · The client addresses the identified issues and provides feedback · Auditors verify the fixes and update their statuses with supporting evidence · A re-audit report is generated and shared with the client OBJECTIVE: VALIDATE THE FIXES AND REASSESS THE CODE TO ENSURE ALL VULNERABILITIES ARE RESOLVED AND NO NEW VULNERABILITIES ARE ADDED.
Final audit	**Final Code Verification & Public Audit Report:** · Verify the final code version against recommendations and their statuses · Check deployed contracts for correct initialization parameters · Confirm that the deployed code matches the audited version · Issue a public audit report, published on our official GitHub repository · Announce the successful audit on our official X account OBJECTIVE: PERFORM A FINAL REVIEW AND ISSUE A PUBLIC REPORT DOCUMENTING THE AUDIT.

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likehood: High	Critical	High	Medium
Likehood: Medium	High	Medium	Low
Likehood: Low	Medium	Low	Low

Impact

- **High** – Theft from 0.5% OR partial/full blocking of funds (>0.5%) on the contract without the possibility of withdrawal OR loss of user funds (>1%) who interacted with the protocol.
- **Medium** – Contract lock that can only be fixed through a contract upgrade OR one-time theft of rewards or an amount up to 0.5% of the protocol's TVL OR funds lock with the possibility of withdrawal by an admin.
- **Low** – One-time contract lock that can be fixed by the administrator without a contract upgrade.

Likelihood

- **High** – The event has a 50-60% probability of occurring within a year and can be triggered by any actor (e.g., due to a likely market condition that the actor cannot influence).
- **Medium** – An unlikely event (10-20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed to minimize potential risks.
- **Medium** – Recommended to be fixed to enhance security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and effectiveness.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and no longer impact its security.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but it has not been fully resolved.
- **Acknowledged** – The recommended fixes have not yet been implemented, and the finding remains unresolved or does not require code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	1
High	1
Medium	4
Low	6

Findings Statuses

ID	Finding	Severity	Status
C-1	Double Order Attack via Callback Mechanism	Critical	Fixed
H-1	Replay Attack via Balance-Based Nonce	High	Fixed
M-1	Silent Truncation in Permit2 Transfers	Medium	Fixed
M-2	Malicious Signer Can Create Any Order and Spend Approvals	Medium	Fixed
M-3	DoS Attack via Dust Executions	Medium	Fixed
M-4	No Way to Cancel Pending Oracle Change	Medium	Fixed
L-1	Approval Source Ambiguity	Low	Fixed
L-2	Precision Loss in Partial Swaps	Low	Fixed
L-3	Minor Issues	Low	Fixed
L-4	Non-Standard EIP-712 Encoding	Low	Fixed
L-5	Zero Staleness Allows Order Invalidation	Low	Fixed
L-6	Signer Rotation Invalidates Outstanding Orders	Low	Fixed

2. Findings Report

2.1 Critical

C-1	Double Order Attack via Callback Mechanism		
Severity	Critical	**Status**	Fixed in 700d5324

Description

The protocol transfers tokens from maker to taker first, and then calls back to the taker (i. e. msg.sender) to complete the order:

```
uint256 actualTakerAmount =
    filledtakerAmount < order.takerAmount ?
    filledtakerAmount : order.takerAmount;

uint256 balanceBefore =
    order.takerToken.balanceOf(address(order.maker));

ISuperpositionCallback(msg.sender).superpositionCallback(
        order, actualTakerAmount, callback);

uint256 balanceAfter =
    order.takerToken.balanceOf(address(order.maker));

// Check that callback provided enough tokens
if (balanceAfter < balanceBefore + actualTakerAmount) {
    revert ReceivedLessThanMinReturn(
        balanceAfter,
        balanceBefore + actualTakerAmount
    );
}
```

src/SuperpositionVault.sol

A malicious taker can exploit this by fulfilling another open order from the same maker during the callback (e.g., an order in the SuperPosition protocol, 1inch, etc.), causing the balance check to pass for both orders without actually transferring the required tokens in the first one.

Additionally, ERC-777 callbacks may be used for the attack.

Recommendation

We recommend calling safeTransferFrom() for both taker and maker directly in SuperpositionVault.swap() at the very end of the code and additionally protect it with a nonReentrant modifier as a precaution. For example:

```
contract SuperpositionVault {
    ...
    function swap(...) ... {
        ...
        ...
        ICallback(msg.sender).callback(...);
        ...
        ...

        token1.safeTransferFrom(maker, taker, ...);
        token2.safeTransferFrom(taker, maker, ...);

        emit ...;

        // no other code
    }
}
```

2.2 High

H-1	Replay Attack via Balance-Based Nonce		
Severity	High	**Status**	Fixed in c3611a76

Description

The protocol uses only the user's token balance as a nonce to prevent replay attacks:

```
uint256 currentMakerBalance = order.makerToken.balanceOf(order.maker);
if (currentMakerBalance < order.nonceBalance) {
    revert ReplayAttackDetected(currentMakerBalance, order.nonceBalance);
}
```

src/SuperpositionVault.sol

However, once the user's balance increases again (e.g., through receiving tokens from other sources), the same order can be executed multiple times if the deadline allows it. This creates a vulnerability where orders can be replayed whenever the maker's balance condition is satisfied again.

Recommendation

We recommend descending from the OpenZeppelin's Nonces contract:
contracts/utils/Nonces.sol

2.3 Medium

M-1	Silent Truncation in Permit2 Transfers		
Severity	Medium	**Status**	Fixed in c3611a76

Description

The use of `uint160(amount)` in Permit2 transfers may silently truncate large amounts:

```
if (usePermit2) {
    permit2.transferFrom(
        order.maker,
        order.taker,
        uint160(amount),
        address(order.makerToken)
    );
} else {
    order.makerToken.safeTransferFrom(
        order.maker,
        order.taker,
        amount
    );
}
```

`src/SuperpositionVault.sol`

When casting a `uint256` amount to `uint160`, values larger than `2^160 - 1` will be silently truncated, potentially resulting in transferring significantly less than intended. For example, `1 << 161` **will truncate to zero**. However, this can be executed only if the transaction is signed by the signer.

Recommendation

We recommend checking `amount < type(uint160).max`.

M-2	Malicious Signer Can Create Any Order and Spend Approvals		
Severity	Medium	**Status**	Fixed in c3611a76

Description

In the current architecture, makers provide approvals to SuperpositionVault, which are then spent when executing orders that have valid signatures from the signer. The problem is that if the signer is compromised, they can create any order and spend any approval from the SuperpositionVault contract.

For example, suppose a maker has given approval to the contract to sell 1 BTC for 120,000 DAI. In this case, a compromised signer could sign and submit a different order - exchanging 1 BTC for 1 wei and specifying themselves as the taker, thus obtaining 1 BTC for free.

There have been numerous cases where motivated hackers have compromised central signers or even multiple signers, particularly in bridge protocols. Therefore, it is better to use a decentralized architecture where both maker and taker sign the transaction, so that orders cannot be arbitrarily created without the signature of the maker themselves.

Recommendation

We recommend checking signatures from both maker and taker instead of using a central signer.

Client's Commentary:

MixBytes: The implementation of three separate signature verification and deadline checks significantly mitigates the severity of this vulnerability. While the centralized signer architecture still poses a theoretical risk, requiring consensus from three independent signers combined with time-bound orders substantially reduces the attack surface and practical exploitability.

M-3	DoS Attack via Dust Executions		
Severity	Medium	**Status**	Fixed in c3611a76

Description

A griefer can frontrun any transaction with a signed order and execute it with an extremely small `filledtakerAmount`. Other users cannot complete the order because the `nonceBalance` check will fail:

```
uint256 currentMakerBalance = order.makerToken.balanceOf(order.maker);
if (currentMakerBalance < order.nonceBalance) {
    revert ReplayAttackDetected(currentMakerBalance, order.nonceBalance);
}
```

`src/SuperpositionVault.sol`

Recommendation

We recommend descending from `contracts/utils/Nonces.sol` and implementing proper logic for multiple partial fills.

M-4	No Way to Cancel Pending Oracle Change		
Severity	Medium	**Status**	Fixed in 5a59556f

Description

When owners propose and execute an oracle update via proposeUpdateOracle → approve → execute, the new oracle is not applied immediately — it is enqueued with a 1-day grace period (ORACLE_CHANGE_GRACE_PERIOD):

```
function _queueOracleUpdate(IPriceOracle newOracle) private {
    _requireNoPendingOracleChange();
    pendingOracleChange.exists = true;
    ...
}
```

src/SuperpositionVault.sol

However, there is no function to cancel a pending oracle change. If during the grace period it is discovered that the enqueued oracle is broken or malicious, the only way to prevent swaps from executing against it is to freeze the entire contract via the proposeFreeze → multisig flow.

Even after freezing, the owners must wait for the grace period to expire so that the broken oracle is applied (via _applyOracleChangeIfReady), then propose a new correct oracle (which triggers another grace period), wait again, and only then unfreeze trading. This results in **1-2 days of complete trading downtime** — a significant disruption for users.

Recommendation

We recommend adding a function that allows the owners to cancel a pending oracle change, clearing the pendingOracleChange state.

2.4 Low

L-1	Approval Source Ambiguity		
Severity	Low	**Status**	Fixed in c3611a76

Description

The protocol allows the executor to choose between Permit2 and direct token approval without this choice being part of the signed order:

```
if (usePermit2) {
    permit2.transferFrom(
        order.maker,
        order.taker,
        uint160(amount),
        address(order.makerToken)
    );
} else {
    order.makerToken.safeTransferFrom(
        order.maker,
        order.taker,
        amount
    );
}
```

src/SuperpositionVault.sol

This creates ambiguity about which approval source will be used. If a user provides unlimited approval to Permit2 and limited approval to the vault, expecting the limited approval to be used first, the use of Permit2 may be unexpected.

Recommendation

We recommend including the approval method (usePermit2 flag) in the order hash to ensure the maker explicitly specifies which approval mechanism should be used.

L-2	Precision Loss in Partial Swaps		
Severity	Low	**Status**	Fixed in c3611a76

Description

The protocol calculates the maker amount using division that can result in precision loss:

```
uint256 amount = filledtakerAmount < order.takerAmount
    ? (order.makerAmount * filledtakerAmount) / order.takerAmount
    : order.makerAmount;
```

src/SuperpositionVault.sol

For example, with makerAmount = 1.0 WBTC (1e8 wei), takerAmount = 120,000.00 DAI (120_000e18), and filledtakerAmount = 0.0011 DAI (i.e. takerAmount / makerAmount - 1), the calculated partial amount results in zero.

Recommendation

We recommend implementing minimum fill amount validation: ensure that filledtakerAmount is greater than several wei and also represents some minimum percentage of the total amount.

L-3	Minor Issues		
Severity	Low	**Status**	Fixed in c3611a76

Description

1. **Inconsistent token validation**: The check `if (order.makerToken == order.takerToken) revert InvalidSender();` should revert with a token-pair specific error.
2. **Duplicate deadline checks**: The codebase has both `<=` and `<` deadline checks - one should be standardized.

Recommendation

- Use appropriate error messages for token pair validation
- Standardize deadline comparison operators

L-4	Non-Standard EIP-712 Encoding		
Severity	Low	**Status**	Fixed in 5a59556f

Description

OrderHelper.hashSingleOrder() and OrderHelper.DOMAIN_SEPARATOR() implement non-standard EIP-712 encoding for fields declared in the type string as uint64, address, and bool.
Specifically, the contract encodes values through bytesN casts followed by extension to bytes32, for example:

- bytes32(bytes8(payload.deadline)) for uint64 deadline
- bytes32(bytes20(payload.maker)) for address maker
- bytes32(bytes20(payload.taker)) for address taker
- bytes32(bytes20(address(payload.makerToken))) for address makerToken
- bytes32(bytes20(address(payload.takerToken))) for address takerToken
- bytes32(bytes1(payload.usePermit2 ? 1 : 0)) for bool usePermit2
- bytes32(bytes20(cont)) for address verifyingContract in DOMAIN_SEPARATOR()

Such conversion yields a left-aligned representation (bytesN semantics), whereas the standard EIP-712 encoder (MetaMask, ethers.js, etc.) for types uint64/address/bool uses ABI/EIP-712-compatible 32-byte encoding (right-aligned for these types).
For example, encoding usePermit2 = true:

- Current: bytes32(bytes1(uint8(1))) = 0x0100000000000000000000000000000000000000000000000000000000000000
- EIP-712 standard: bytes32(uint256(1)) = 0x0000000000000000000000000000000000000000000000000000000000000001

As a result:

- The order structure hash computed by the contract does not match the standard EIP-712 typed-data hash;
- The contract's DOMAIN_SEPARATOR also does not match the standard EIP-712 domain separator for the declared domain EIP712Domain(string name,string version,uint256 chainId,address verifyingContract);
- Signatures obtained via standard EIP-712 wallets/libraries may fail validation in the contract.

- src/SuperpositionVault.sol
- src/SuperpositionVault.sol

Recommendation

We recommend using standard right-aligned encoding for all fields to ensure compatibility with EIP-712.

L-5	Zero Staleness Allows Order Invalidation		
Severity	Low	**Status**	Fixed in 8da54dd8

Description

Users can set `maxStaleness` to `0` in `setUserConfig()`, which causes all oracle price checks to fail (since `block.timestamp - updatedAt > 0` is always true for any real oracle), effectively allowing users to pause/cancel all their orders as a workaround for order cancellation.

Recommendation

We recommend enforcing a minimum `maxStaleness` value (e.g., `1 seconds` or `1 minutes`) to prevent users from setting it to zero, or document this behavior as an intended feature for order invalidation.

L-6	Signer Rotation Invalidates Outstanding Orders		
Severity	Low	**Status**	Fixed in 5a59556f

Description

The contract stores only one active signer set. When a new set is proposed and executed, the previous addresses are overwritten. The swap() function validates every order signature strictly against the *current* signer trio, without any versioning information in the order hash. Consequently, any order that was signed under a previous signer set becomes unverifiable after rotation, permanently cancelling all outstanding liquidity tied to those orders. Because signer rotation is an expected governance/maintenance operation, this may pose a denial-of-service scenario.

Recommendation

We recommend implementing a grace period during which both the old and new signer sets are accepted in the swap() function. Note, that old signers must not be accepted for proposal and approval mechanics.

3. About MixBytes

MixBytes is a leading provider of smart contract auditing and blockchain security research, helping Web3 projects enhance their security and reliability.

Since its inception, MixBytes has been dedicated to safeguarding innovation in **DeFi** through rigorous audits and cutting-edge technical research.

Our team brings together experienced engineers, security auditors, and blockchain researchers with deep expertise in smart contract security and protocol design.

With years of proven experience in Web3, MixBytes combines in-depth technical excellence with a proactive, security-first approach.

Why MixBytes

- **Proven Track Record:** Trusted by leading blockchain protocols such as **Lido**, **Aave**, **Curve**, **1inch**, **Fluid**, **Gearbox**,**Resolv**, and others. MixBytes has successfully secured billions of dollars in digital assets.
- **Technical Expertise:** Our auditors and researchers hold advanced degrees in cryptography, cybersecurity, and distributed systems, backed by years of hands-on experience.
- **Innovative Research:** MixBytes actively contributes to blockchain security research and open-source tools, sharing insights with the global community.

Our Services

- **Smart Contract Audits:** Comprehensive security assessments of DeFi protocols to detect and mitigate vulnerabilities before deployment.
- **Blockchain Research:** In-depth technical studies, economic and protocol-level modeling for Web3 projects.
- **Custom Security Solutions:** Tailored frameworks and advisory for complex decentralized systems and blockchain ecosystems.

Contact Information

https://mixbytes.io/

https://github.com/mixbytes/audits_public

https://x.com/MixBytes

hello@mixbytes.io